

Application of Iterative and Decision-Making Constructs to Shapes and Physics using Modular Programming in C

Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs. Then develop, compile, and execute a **menu-driven modular C project** that calculates the geometrical measurements and physical properties of selected solids. Test the program using the prepared test cases.

[**Note:** Use only fundamental data types, functions with call by value, selection statements, and loops. Do not use recursion, arrays, pointer variables, pointer parameters, structures, unions, enumerations, typedef, or functions from math.h. Assume that all input values and calculated results are within the ranges supported by their respective data types.]

Operations

- Find the total surface area and volume of a sphere [*double sphere_area(double radius), double sphere_volume(double radius)*]
- Find the total surface area and volume of a cylinder [*double cylinder_area(double radius, double height), double cylinder_volume(double radius, double height)*]
- Find the total surface area and volume of a cone [*double cone_area(double radius, double height), double cone_volume(double radius, double height)*]
- Find the total surface area and volume of a cuboid [*double cuboid_area(double length, double breadth, double height), double cuboid_volume(double length, double breadth, double height)*]
- Find the base area of cylinder and cone [*double circular_base_area(double radius)*]
- Find the base area of cuboid [*double cuboid_base_area(double length, double breadth)*]
- Calculate the mass of a solid from its density and volume [*double calculate_mass(double density)*]
- Calculate the weight of a solid [*double calculate_weight()*]
- Calculate the gravitational potential energy of a solid at a given elevation [*double potential_energy(double elevation)*]
- Calculate the pressure exerted by a cylinder, cone, or cuboid resting on its base [*double calculate_pressure(double density)*]
- Calculate the buoyant force on a completely submerged solid [*double buoyant_force(double fluid_density)*]
- Determine whether a completely immersed solid sinks, floats, or remains suspended [*int determine_buoyancy(double solid_density, double fluid_density)*]

Organization of the project

Module	Functions/Responsibility
main.c	Displays the main menu, obtains common inputs, calls the required functions, and displays results
geometry.c	Calculates surface area, volume, base area, and slant height
physics.c	Obtains the solid type and required dimensions for physics operations and performs the corresponding calculations
math_utils.c	Calculates square root iteratively using the Newton-Raphson method
solid.h	Contains the header guard, symbolic constants, and external function declarations

Header File Outline

- Use `#ifndef`, `#define`, and `#endif` to create a header guard.
- Define `PI` as 3.1416, `GRAVITY` as 9.8, and `TOLERANCE` as 0.000001.
- Define the solid choices `SPHERE`, `CYLINDER`, `CONE`, and `CUBOID` with the values 1, 2, 3, and 4, respectively.
- Define the buoyancy results `FLOATS`, `SUSPENDED`, and `SINKS` with the values -1, 0, and 1, respectively.
- Include the external declarations of all functions defined in *geometry.c*, *physics.c*, and *math_utils.c*.

Outline of main()

- Use a **do-while** loop to repeat the menu until the Exit option is selected.
- Display the menu: (1) Surface area, (2) Volume, (3) Base Area, (4) Mass, (5) Weight, (6) Gravitational potential energy, (7) Pressure on the base, (8) Buoyant force, (9) Buoyancy behavior, and (10) Exit.
 - For operations involving geometry, display a **submenu** to select the solid.
 - **Supported Solids:** Sphere, Cylinder, Cone, Cuboid
- Validate each input immediately after it is obtained, whether it is obtained in main() or another function. Solid choices must be within the specified range. Dimensions and densities must be greater than zero, and elevation must not be negative. If an invalid value is entered, display an appropriate error message and request the value again.
- For the Base Area option, permit only cylinder, cone, and cuboid. If the user selects sphere, display an appropriate message and return to the main menu.
- Use SI units throughout the project. Enter dimensions and elevation in metres and densities in kilograms per cubic metre. Display area in square metres, volume in cubic metres, mass in kilograms, weight and buoyant force in newtons, pressure in pascals, and potential energy in joules.
- Use *switch-case* to call the corresponding function and display the result.
- Do not repeat geometrical or physical formulas in *main()*.

Outline of geometry.c

- sphere_area()
 - Calculate and return $4 * PI * radius * radius$.
- sphere_volume()
 - Calculate and return $(4.0 / 3.0) * PI * radius * radius * radius$.
- cylinder_area()
 - Calculate and return $2 * PI * radius * (radius + height)$.
- cylinder_volume()
 - Calculate and return $PI * radius * radius * height$.
- cone_area()
 - Calculate the *slant_height* as the square root of $radius * radius + height * height$ by calling *calculate_square_root()*.
 - Calculate and return $PI * radius * (radius + slant_height)$.
- cone_volume()
 - Calculate and return $(1.0 / 3.0) * PI * radius * radius * height$.
- cuboid_area()
 - Calculate and return $2 * (length * breadth + breadth * height + length * height)$.
- cuboid_volume()
 - Calculate and return $length * breadth * height$.
- circular_base_area()
 - Calculate and return $PI * radius * radius$.
 - Use it for cylinders and cones.
- cuboid_base_area()
 - Calculate and return $length * breadth$.
- **Note:** As the ideal geometrical base area for a sphere is not defined by the given dimensions, it cannot be calculated.

Outline of physics.c

- calculate_mass()
 - Get the type of solid (sphere/cone/cylinder/cuboid).
 - Use appropriate function to calculate the volume of the solid. i.e. *sphere_volume()* or *cone_volume()* or *cylinder_volume()* or *cuboid_volume()*.

- Use $mass = density * volume$ and return the mass in kilograms.
- `calculate_weight()`
 - Obtain mass from `calculate_mass()`.
 - Use $weight = mass * GRAVITY$ and return the weight in newtons.
- `potential_energy()`
 - Obtain mass from `calculate_mass()`.
 - Use $potential\ energy = mass * GRAVITY * elevation$ and return the energy in joules.
- `calculate_pressure()`
 - Get the type of solid (cone/cylinder/cuboid).
 - Obtain `contact_area` from `circular_base_area()` for cylinder or cone, and from `cuboid_base_area()` for cuboid.
 - Obtain `volume` from appropriate function `cone_volume()` or `cylinder_volume()` or `cuboid_volume()`.
 - Use $pressure = (density * volume * GRAVITY) / contact_area$ and return the pressure in pascals.
 - **Note:** Do not calculate pressure for a sphere because the ideal geometrical contact area is not defined by the given dimensions.
- `buoyant_force()`
 - Get the type of solid (sphere/cone/cylinder/cuboid).
 - Obtain `volume` from appropriate function `sphere_volume()` or `cone_volume()` or `cylinder_volume()` or `cuboid_volume()`.
 - Use $buoyant\ force = fluid_density * volume * GRAVITY$ and return the force in newtons.
- `determine_buoyancy()`
 - Return `SINKS` if `solid_density` is greater than `fluid_density`.
 - Return `FLOATS` if `solid_density` is less than `fluid_density`.
 - Return `SUSPENDED` if the two densities are equal.
 - Display the corresponding message in `main()`.

Outline of `math_utils.c`

- `calculate_square_root()`
 - Do not use `sqrt()`, `fabs()`, or any other function from `math.h`.
 - If `number` is negative, return -1 to indicate that its real square root cannot be calculated.
 - If `number` is zero, return 0.
 - For a positive number, initialize `old_value` with `number`.
 - Use a *do-while* loop to perform the following steps:
 - Calculate a better approximation using
 - $new_value = 0.5 * (old_value + number / old_value)$
 - Calculate the difference between the new and previous approximations:
 - $difference = new_value - old_value$
 - If *difference* is negative, change it to its positive value.
 - Assign `new_value` to `old_value` so that the new approximation is used in the next iteration.
 - Continue the loop while *difference* is greater than or equal to `TOLERANCE`.
 - Return `new_value` as the approximate square root of `number`.